An Adaptive Multi-Agent System for Student Competency Analysis Across Cognitive Domains

1st Sreeja Sravanam
Dept. of CSE (Data Science)
Prasad V. Potluri Siddhartha Institute of Technology
Vijayawada, India
sreeja@pvpsiddhartha.ac.in

2nd Dunna Sushanth Teja
Dept. of CSE (Data Science)
Prasad V. Potluri Siddhartha Institute of Technology
Vijayawada, India
22501A4412@pvpsit.ac.in

3rd Katta Jatadhara Sai
Dept. of CSE (Data Science)
Prasad V. Potluri Siddhartha Institute of Technology
Vijayawada, India
22501A4425@pvpsit.ac.in

4th Sujaya Raghavi Pinapala
Dept. of CSE (Data Science)
Prasad V. Potluri Siddhartha Institute of Technology
Vijayawada, India
22501A4459@pvpsit.ac.in

***Abstract*—Based on the traditional educational assessment, the real capability of learners is not reflected as the same questions are given to everyone despite personal competency levels. In this paper, a Multi-Agent MCQ-based Student Competency Analysis System is introduced, which assesses student responses and measures learner competency in various cognitive areas with the full pipeline of the LLM. Based on the multi-agent competency-building model of Outay *et al.* [1], we apply the Assess–Predict–Oversee–Transit (APOT) model to the question-based assessment scenario and redefine its agents, the Learning Style Assessment Agent (LSAA), Stage of Learning Monitoring Agent (SOLMA), Stage of Learning Prediction Agent (SOLPA), and Stage of Learning Trans The system is coordinated by the CrewAI, which implements three specialized agents, which run on the Google Gemini 2.5 Flash: an MCQ Evaluator Agent, a Difficulty Predictor Agent, and an Adaptive MCQ Overseer Agent. A dual-layer architecture is used to guarantee UI responsiveness with synchronous rule-based fallback and a background multi-agent pipeline offers AI-based competency analysis. The LLM generates questions dynamically, avoiding the use of pre-calibrated Item Response Theory models and fixed item banks. The design level analysis illustrates logical consistency in three cognitive areas, namely: Reasoning, Aptitude and Verbal Ability.**

***Index Terms*—adaptive assessment, student competency analysis, multi-agent system, CrewAI, Gemini, LLM-based adaptation, cognitive domains, APOT model**

I. Introduction

Conventional testing of education follows the one-size-fits-all approach, where all students are given the same questions, irrespective of personal abilities. This method presupposes the homogeneity of learning ability and pace among learners in an inherently unequal way in real-life learning settings. Consequently, high-achieving students will find the test too simple and effortless, whereas the low-achieving students will be unable to cope with it properly, becoming frustrated and disinterested. Therefore, these systems do not represent the actual capability of the learners on either end of the performance axis and offer minimal actionable information to instructors about the strengths, weaknesses, and learning paths of individual learners [2].

In addition, the majority of the current assessment systems give little or general feedback that fails to assist the students in knowing their areas of strength and weakness. This non-personalization impedes proper measurement of competencies, makes students less engaged, and restricts the learning efficiency [9]. Assessments cannot be dynamic unless they are constantly altered according to the performance of the students and this will not capture the dynamic aspects of learning as it happens constantly in the learning process [10].

Computerized Adaptive Testing (CAT) helps to overcome these drawbacks by making question difficulty adapt to continued responses [3]. Nevertheless, the majority of CATs are based on the Item Response Theory (IRT), which involves large banks of pre-calibrated items and parameters that have been statistically validated. This restricts its applicability to smaller settings and complicates question selection in ways that are difficult for educators to interpret.

Recent progress on Large Language Models (LLMs) provides another way: by encoding adaptation logic in intelligent agents. Agents operating on LLMs are able to reason holistically on the basis of correctness and response time, utilizing natural language adaptation principles to contextually decide on difficulty, which directly feeds into competency analysis.

The APOT (Assess-Predict-Oversee-Transit) model proposed by Outay et al. [1], managed by a Multi-Agent System (MAS) for competency building, comprises the Learning Style Assessment Agent (LSAA), Stage of Learning Monitoring Agent (SOLMA), Stage of Learning Prediction Agent (SOLPA), and Stage of Learning Transition Agent (SOLTA). Although their framework offers a solid basis, it focuses on augmented reality (AR)-based vocational training and relies on deterministic KPI metrics. We adapt the cyclic steps and separation of agent roles of the APOT model to an entirely question-driven adaptive student competency analysis setting

based on LLMs.

The proposed Adaptive Multi-Agent Student Competency Analysis System makes the following contributions:

1) A three-agent CrewAI pipeline running on Gemini 2.5 Flash, mapping the APOT agents (SOLMA, SOLPA, SOLTA), replacing formula-driven scoring with LLM-native contextual reasoning to assess student competency across cognitive domains.
2) A two-layered difficulty adaptation mechanism combining a synchronous rule-based fallback with asynchronous multi-agent LLM analysis and LLM-driven question generation, removing reliance on fixed item banks.
3) An interpretable, prompt-engineered adaptation mechanism that eliminates IRT pre-calibration without sacrificing responsiveness or explainability across three cognitive domains.

II. Related Work

A. Multi-Agent Systems in Education

Multi-Agent Systems (MAS) have become a popular educational technology topic of discussion [7]. Outay *et al.* introduced the APOT model that has four agents: the **LSAA** (Learning Style Assessment Agent), which measures the preferences of the learner in the modality; the **SOLMA** (Stage of Learning Monitoring Agent), which assesses the current performance of the learner; the **SOLPA** (Stage of Learning Prediction Agent), which predicts the potential of the learner; and the **SOLTA** (Stage of Learning Transition Agent), which manages transitions between learning stages. To monitor progress, these agents calculate the Learning Index (LI) and Competency Index (CI). Nevertheless, their framework focuses on vocational AR training, which restricts its use to digital question-based assessment [1]. Our work adapts the APOT philosophy to a question-oriented environment by substituting quantitative KPIs with LLM-based reasoning to analyze competencies.

B. Computerized Adaptive Testing

Since the 1970s, CAT has been extensively studied and is generally considered a powerful approach to personalized assessment [2]. CAT systems are usually founded on Item Response Theory (IRT), in which each question is associated with parameters such as difficulty, discrimination, and guessing probability [3]. Although statistically rigorous, IRT-based CAT systems require large, well-calibrated item banks, which are both costly and time-intensive to construct [4]. IRT models are also difficult for educators to interpret due to their probabilistic nature. Our method substitutes IRT with LLM-based agents that deliberate over observable performance signals — correctness and response time — using clear rules written in natural language, enabling easy deployment without sacrificing adaptive responsiveness.

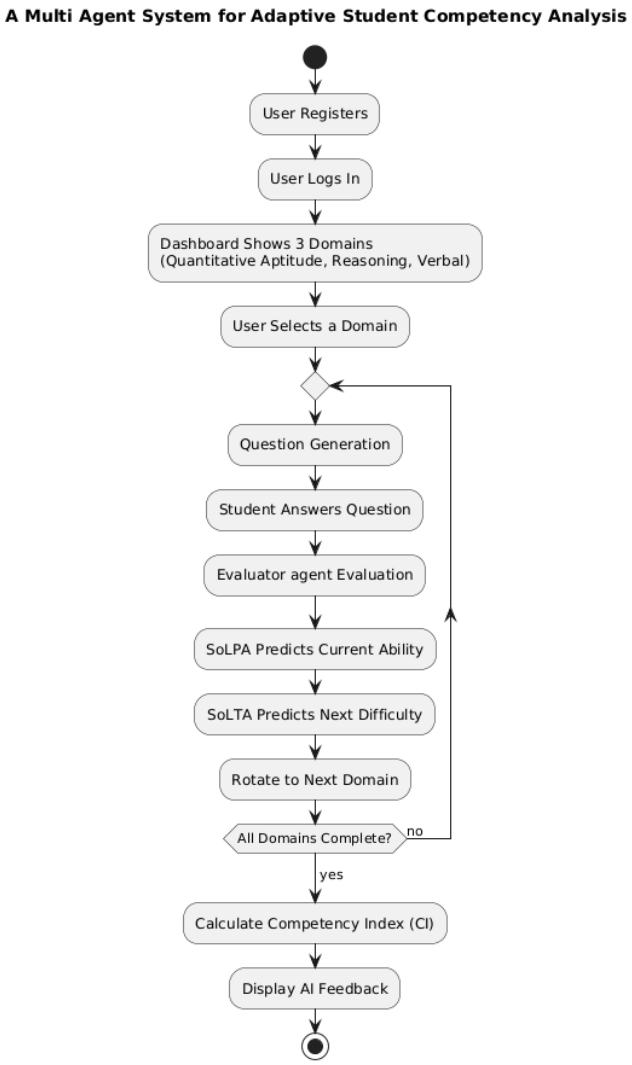

Fig. 1. Three-agent CrewAI pipeline implementing the APOT model.

III. System Architecture

A. Architecture Overview

The system is a full-stack web application using Django 4.2 (Python) with MySQL, organized into five layers. The **Presentation Layer** (Bootstrap 5.3, vanilla JavaScript) handles question display, answer input, and a 20-minute countdown timer, with a fullscreen enforcement gate via the browser Fullscreen API and `navigator.sendBeacon` for auto-submission on tab closure. The **Controller Layer** (Django views) coordinates the test lifecycle, implements the synchronous rule-based difficulty fallback, and spawns asynchronous agent threads. The **AI/Agent Pipeline Layer** runs the CrewAI pipeline (MCQ Evaluator Agent → Difficulty Predictor Agent → Adaptive MCQ Overseer Agent) in a background daemon thread communicating with Gemini 2.5 Flash. The **Question Generation Layer** invokes Gemini via the `google-generativeai` SDK to generate MCQs

on-the-fly with a strict JSON schema. The Data Layer is based on three models, which are backed by MySQL: `StudentTestResult`, `QuestionPerformance` , and `AdaptiveLearningHistory`.

B. Assessment Workflow

The system is based on a per-question adaptive cycle through 20 questions per session. For each question, the student's selected answer and response time are captured. The MCQ Evaluator Agent determines correctness; the Difficulty Predictor Agent classifies current question difficulty; the Adaptive MCQ Overseer Agent synthesizes results and recommends the next difficulty level. Three cognitive domains are supported: Reasoning, Aptitude (aliases: Quantitative Aptitude, Quant), and Verbal Ability (aliases: Verbal, English), with topic normalization enforcing a controlled vocabulary.

C. APOT Model Adaptation

The original APOT model [1] follows a cyclic Assess, Predict, Oversee, and Transit process managed by LSAA, SOLMA, SOLPA, and SOLTA. In our adaptation, the *Assess* stage maps to the MCQ Evaluator Agent (implementing the SOLMA role) by monitoring answer correctness. The *Predict* stage maps to the Difficulty Predictor Agent (implementing the SOLPA role) by classifying question difficulty. The *Oversee* and *Transit* stages jointly map to the Adaptive MCQ Overseer Agent (implementing the SOLTA role) by synthesizing analyses and determining transitions. The LSAA function is embodied by domain-aware question generation and multi-domain competency tracking.

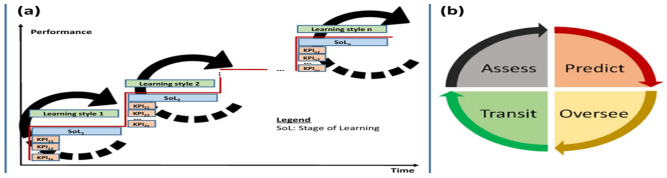

Fig. 2. The APOT (Assess–Predict–Oversee–Transit) cyclic learning model from Outay *et al.* [1], showing four stages managed by LSAA, SOLMA, SOLPA, and SOLTA agents. Our system reinterprets these as LLM-powered CrewAI modules.

D. Dual-Layer Difficulty Adaptation

One of the most important innovations decouples UI responsiveness from the more computationally intensive multi-agent pipeline. The **Synchronous Layer** is a rule-based mapping that enforces a deterministic difficulty transition immediately when an answer is submitted: Easy → Medium → Hard → Hard → Hard → Medium → Easy → Easy. This difficulty determines the next question, which is returned in the JSON response to update the UI in under a second. The **Asynchronous Layer** spawns a daemon thread that runs the entire CrewAI pipeline: MCQ Evaluator Agent → Difficulty Predictor Agent → Adaptive MCQ Overseer Agent. The AI analysis results are persisted to `AdaptiveLearningHistory`, and `final_difficulty` is updated accordingly. This thread maintains a database connection enclosed in a `finally` block to prevent connection leakages.

IV. Multi-Agent Framework

The agents are driven by Gemini 2.5 Flash through the CrewAI LLM interface (temperature = 0.7), running sequentially with shared context passed between tasks.

A. MCQ Evaluator Agent

The MCQ Evaluator Agent takes over the role of SOLMA (Stage of Learning Monitoring Agent), which is similar to the Assess stage of APOT. Just like the initial SOLMA, this agent monitors the quality of the immediate response of the student, in this case, the performance at that specific point in time. It works like a fair and accurate MCQ examiner: the text of the question, the correct response, the response (answer) the student selected, and the amount of time taken all are inputted and it determines whether the answer is correct or not, and also gives a one-line response. It is set to allow no delegation, i.e., a setting of allow delegation=False, in order to prevent the factual judgement being mixed up with difficulty reasoning.

The input block that is received by the agent is:

- Question text.
- The right answer.
- The response that the student has given.
- Time in seconds.

It is created to generate a binary verdict of correctness with a brief justification, the factual basis of further agents.

B. Difficulty Predictor Agent

The Difficulty Predictor Agent takes the place of SOLPA (Stage of Learning Prediction Agent), which is the Predict stage of the APOT model. It assigns a level of difficulty to the question at hand as Easy, Medium or Hard depending on the recommendation of the MCQ Evaluator Agent and the response time taken by the student. The following heuristics are coded in the backstory: Fast + Correct Easy; Slow + Correct Medium; Wrong Hard. This is in line with the fact that response time and correctness is a behavioral proxy to perceived difficulty [2]. There are no hard-coded thresholds, the notion of what constitutes slow and fast is contextually sensitive and it is up to the reasoning of the LLM.

C. Adaptive MCQ Overseer Agent

Adaptive MCQ Overseer Agent is the agent that plays the role of SOLTA (Stage of Learning Transition Agent) and embodies the stages of APOT framework, namely, Transit and Oversee. Unlike the original SOLTA that dictates between competency levels, this agent chooses to move between difficulty levels at his own discretion in relation to the emerging competency in the student. It accepts the output of the MCQ Evaluator Agent and Difficulty Predictor Agent and returns: (1) an agreed-upon difficulty, and (2) a proposed difficulty. Its backstory is a code of rules of adaptation based on the Zone of Proximal. Development [9] and Bloom's Taxonomy:

In the case of a correct answer:

- Easy -Next: Medium (challenge further)
- Medium-Next: Hard (to the next level)
- Hard Soft (consolidate mastery) Next: Hard or Medium

When the answer is not correct:

- Hard → Next: Medium (descend one level)
- Medium → Next: Easy (step down one level)
- Easy → Next: Easy (remain at the same level to build confidence)

It is configured with `allow_delegation=True` so that the agent may seek further clarification from preceding agents. Its output adheres to a structured natural language format parsed via line-by-line keyword matching, with Medium used as the default fallback:

```
Question Difficulty: [Easy/Medium/Hard]
Next Question Difficulty: [Easy/Medium/Hard]
Reasoning: [explanation]
Recommendation: [additional actions]
```

D. CrewAI Orchestration

A `Crew` of the three agents is assembled with `Process.sequential`, `memory=False`, and `max_rpm=10`. Task 3 (Adaptive MCQ Overseer Agent) accesses the outputs of the MCQ Evaluator Agent and Difficulty Predictor Agent via `context=[task1, task2]`, and `crew.kickoff()` executes the pipeline and returns the output of the Adaptive MCQ Overseer Agent. The pipeline is depicted in Fig. 1.

V. Methodology

A. Input Representation

For each MCQ response, the system constructs a structured input string:

```
Question: <question text>
Correct Answer: <correct option>
Student Selected Answer:Student choice:
Time Taken by Student: <seconds> seconds
```

This reflects the truth to fact and efficiency of response. Per-question timing is measured client-side via `Date.now()` differencing, with a default of 30 seconds when omitted.

B. Agent Prompt Design

Each agent's behavior in CrewAI is governed by three natural language fields: `role`, `goal`, and `backstory`. These fields function as the system prompt for the underlying Gemini model, shaping its reasoning approach for each task. The role defines the agent's identity within the APOT pipeline, the goal specifies its objective, and the backstory provides the contextual knowledge and heuristics the agent should apply.

For the Difficulty Predictor Agent, the backstory explicitly encodes the time-correctness heuristic: "Fast + Correct = Easy question. Slow + Correct = Medium. Wrong = Hard." This concise rule set mirrors the psychometric intuition behind time-aware adaptive testing [2] and eliminates the need for IRT parameter estimation.

For the Adaptive MCQ Overseer Agent, the backstory encodes the full bidirectional transition table (increase on success, decrease on failure) aligned with the Zone of Proximal The concept of development [9] that keeps students. within their optimal challenge zone and enables accurate competency stage determination.

C. Robustness Mechanisms

The system employs multiple fallback strategies to ensure uninterrupted operation. On API key failure, the pipeline returns a canned "Medium"/"Medium" result with reasoning "Using default difficulty due to API key issue," ensuring the test session is never interrupted. The question generation module retries transient errors (HTTP 503, 429, 500, and messages containing "UNAVAILABLE", "overloaded", or "high demand") up to 3 times with exponential-like backoff of $(attempt+1)\times 2$ seconds. If all retries and JSON parsing fail, a stub fallback question is returned to prevent test interruption. The background CrewAI thread explicitly closes its thread-local Django database connection in a `finally` block to prevent connection leaks from the daemon thread performing ORM writes.

D. Session Management and Test Integrity

Each test session is identified by a UUID (`assessment_id`) linking the transient session state stored in `request.session` with the durable `StudentTestResult` record in MySQL. The session stores the dynamically generated question list, current index, answer history, skipped indices, difficulty progression, and timer state. Status transitions follow a controlled state machine: `in_progress` transitions to `completed` on explicit submission, to `auto_submitted` when `navigator.sendBeacon` fires on tab closure, or to `expired` when the student clicks the Exit button.

An enforcement gate blocks the test UI with an overlay in front of fullscreen mode until the student specifically requests fullscreen mode. A skip-question revisit scheme enables students to jot a question: the indices that are skipped are remembered and revisited when the linear is re-read. pass completes, interleaving forward progress with deferred-item completion. The 20-minute timer is enforced both on the frontend via a JavaScript countdown and on the backend by recomputing elapsed time from the stored `start_time` on each request.

VI. System Analysis and Discussion

A. Role Separation in Agents and Pipeline Integrity

The separation of concerns in the three-agent design directly reflects the staged reasoning of the APOT model [1]. The MCQ Evaluator Agent performs only factual comparison, ensuring that correctness determination remains independent of difficulty reasoning. The Difficulty Predictor Agent applies psychometric heuristics solely to the question at hand, without considering adaptation goals. The Adaptive MCQ Overseer Agent combines correctness, difficulty prediction, and adaptive

rules to generate actionable transition recommendations that reflect the student's competency path. This layered design avoids conflation of concerns and makes the reasoning of each agent independently auditable. The mapping is summarized in Table 1.

TABLE I
APOT PHASE TO CREWAI AGENT MAPPING.

APOT Phase	APOT Agent	CrewAI Agent	Function
Assess	SOLMA	MCQ Evaluator Agent	Answer correctness verdict
Predict	SOLPA	Difficulty Predictor Agent	Difficulty classification
Oversee	SOLTA	Adaptive MCQ Overseer Agent	Synthesis and confirmation
Transit	SOLTA	Adaptive MCQ Overseer Agent	Next difficulty recommendation

B. Adaptive Rule Coverage

Table 2 lists all six correctness-difficulty combinations handled by the Adaptive MCQ Overseer Agent.

TABLE II
ADAPTIVE DIFFICULTY TRANSITION RULES APPLIED BY THE ADAPTIVE MCQ OVERSEER AGENT.

Correct?	Current	Next	Rationale
Yes	Easy	Medium	Progressive challenge
Yes	Medium	Hard	Advance to next level
Yes	Hard	Hard	Maintain mastery
No	Hard	Medium	Step down one level
No	Medium	Easy	Step down one level
No	Easy	Easy	Reinforce confidence

The rules eliminate oscillation (Hard + Correct $\rightarrow$ Hard) and floor trapping (Easy + Correct $\rightarrow$ Medium).

C. Behavioral Trace

Table 3 shows the anticipated outputs under typical performance conditions.

The trace indicates that the system correctly increases the difficulty of students that respond quickly and correctly, which means that they have mastered easier content. Slow correct students are put in Medium difficulty which indicates effortful but effective performance. Students with wrong answers, either fast or slow are classified as having weakness with Hard questions and thus graded down. This is in line with the Zone of Proximal Development proposed by Vygotsky (1978) which keeps the students within the zone of what they can already do without causing them to become frustrated or bored.

TABLE III
EXPECTED OUTPUT OF THE APOT PIPELINE UNDER TYPICAL SCENARIOS.

Scenario	Correct?	Time	Predictor	Overseer
Fast + Correct	Yes	10s	Easy	Medium
Slow + Correct	Yes	50s	Medium	Hard
Fast + Incorrect	No	10s	Hard	Medium
Slow + Incorrect	No	55s	Hard	Medium

D. Competency Data Models and Analytics

The system also offers longitudinal competency tracking along with per-question-based adaptation. To compute a Competency Index (CI) shown on the dashboard, the harmonic weights of the ten most recent assessments are computed (weight = 1/rank) so that recovering performance more recently gets a higher weight. Subdomain accuracy (Aptitude, Reasoning, and Verbal) is assessed individually to determine areas of weakness and give specific feedback. The leaderboard has the following competency bands: Expert (at least 80), Advanced. (≥ 65), Intermediate (≥ 50), and Beginner (< 50).

The persistence layer consists of three related models which offer a full audit trail: StudentTestResult contains: user, topic, initial and final difficulty, score percentage (computed automatically), status, time taken, and the complete CrewAI reasoning and recommendation text.

E. LLM-Native Adaptation: Benefits and Trade-offs

A key design decision is that all the reasoning of difficulty is delegated to the LLM agents instead of deterministic formulas. This has a number of advantages. First, the rules of adaptation are written in only natural language, and thus can be read and edited without re-writing scoring code, leading to better maintainability. Second, the agents are able to encode contextual subtleties in borderline cases involving time limits, which are impossible to represent as strict numeric rules; the Difficulty Predictor Agent makes judgments about what is considered fast or slow with respect to local circumstances as opposed to having a second hard-coded limit. Third, the entire line of reasoning is continuously in view in the agent output, which offers educators a natural language description of every difficulty decision, which directly contributes to the explainability goals of the APOT framework [1]. Fourth, the system can be deployed to any domain without prior construction of item banks, as no pre-calibrated IRT parameters are required. Fifth, LLM-driven question generation eliminates the traditional bottleneck of domain bootstrapping: new domains require only a subject name and difficulty level, with no pilot testing or statistical calibration needed.

The primary trade-off is LLM output non-determinism, where identical inputs may occasionally produce slightly different outputs. This is mitigated by the structured output format enforced through task descriptions, line-by-line

keyword parsing with a hard default of Medium on failure, and most importantly by the synchronous rule-based fallback layer, which ensures that real-time question selection remains deterministic.

VII. Conclusion and Future Work

This paper presented an Adaptive Multi-Agent Student Competency Analysis System that operationalizes the APOT framework [1] in a fully LLM-driven adaptive testing context. By mapping the MCQ Evaluator Agent, Difficulty Predictor Agent, and Adaptive MCQ Overseer Agent—corresponding to the SOLMA, SOLPA, and SOLTA roles respectively—to three CrewAI agents powered by Gemini 2.5 Flash, the system replaces deterministic scoring and IRT pre-calibration with prompt-engineered LLM reasoning over answer correctness and response time.The LSAA (Learning Style Assessment Agent) is achieved by question generation by domain awareness on Reasoning, Aptitude, and Verbal Ability.

The dual-layer structure, which has a synchronous rule-based fallback to achieve real-time responsiveness and an asynchronous multi-agent AI pipeline to provide rich post-assessment competency information, shows that adaptive student competency analysis can be scaled to production scale and coexist with computationally intensive LLM pipelines. The dynamic question generation driven by the LLM can remove the conventional need of pre-curated statistically validated item banks. Design-level analysis ensures full coverage of the transition space, behaviorally consistency and a natural language audit trail of all the difficult decisions. The system has shown that multi-agent architectures that are native to the LLM support adaptive competency analysis logic, which is both interpretable and can be deployed without item banks that need to be pre-calibrated, as well as extendable by simple modification of prompts.

Future research will concentrate on: (1) user studies with various users. students populations to empirically determine the correctness of the difficulty labels of the LLM output to expert annotations; (2) enabling the LSAA functional with an explicit learner-style profiling agent to adjust question format to detected preferences; (3) enabling the session-level memory (set by default to memory=True in CrewAI) to track cross-question performance and plan competency over a longer horizons.

References

[1] F. Outay, N. Jabeur, F. Bellalouna, and T. Al Hamzi, "Multi-agent system-based framework for an intelligent management of competency building," *Smart Learning Environments*, vol. 11, no. 41, pp. 1–18, 2024, doi: 10.1186/s40561-024-00328-3.

[2] D. J. Weiss, "Computerized adaptive testing for effective and efficient measurement in counseling and education," *Measurement and Evaluation in Counseling and Development*, vol. 37, no. 2, pp. 70–84, 2004.

[3] F. M. Lord, *Applications of Item Response Theory to Practical Testing Problems*. Hillsdale, NJ, USA: Lawrence Erlbaum Associates, 1980.

[4] R. K. Hambleton and H. Swaminathan, *Item Response Theory: Principles and Applications*. Boston, MA, USA: Kluwer-Nijhoff, 1985.

[5] J. S. Liang *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[6] C. Conati, A. Gertner, and K. VanLehn, "Using Bayesian networks to manage uncertainty in student modeling," *User Modeling and User-Adapted Interaction*, vol. 12, no. 4, pp. 371–417, 2002.

[7] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, UK: John Wiley & Sons, 2009.

[8] B. S. Bloom *et al.*, *Taxonomy of Educational Objectives: The Classification of Educational Goals, Handbook I: Cognitive Domain*. New York, NY, USA: David McKay, 1956.

[9] L. S. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*. Cambridge, MA, USA: Harvard University Press, 1978.

[10] R. J. Mislevy, L. S. Steinberg, and R. G. Almond, "On the structure of educational assessments," *Measurement: Interdisciplinary Research and Perspectives*, vol. 1, no. 1, pp. 3–62, 2003.

[11] L. Chen *et al.*, "Large language models in education: A review of applications, opportunities, and challenges," *Educational Technology & Society*, vol. 26, no. 4, pp. 1–18, 2023.

[12] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.